Grafo

Banco de dados

Arquitetura de dados do Grafo: diagrama de relacionamentos, dicionário de todas as tabelas, ciclo de vida do documento colaborativo, formato binário, Redis e NATS JetStream, consultas prontas, preparação do PostgreSQL e regras de integridade e segurança.

Copyright (©) 2026 Erik Pimenta
Versão online: https://ravelroot.github.io/Collaborative-Design-and-Diagram-Editor/

Sumário

Visão geral **4**
PostgreSQL é a fonte da verdade 4
Camadas de armazenamento 4
Áreas do modelo 4
Modo local (sem servidor) 5
Diagrama de relacionamentos **6**
Diagrama geral 6
Identidade e equipes 7
Arquivos e permissões 7
Documento colaborativo 8
Comentários, assets, bibliotecas e dados 8
Regras e convenções **9**
Convenções do modelo 9
Integridade 9
Validação antes de gravar 9
Tabelas **10**
users 10
teams 10
team_members 10
projects 11
files 11
file_permissions 11
share_links 12
document_states 12
document_updates 12
versions 13
comments 13
assets 13
library_components 14
data_sources 14
Tipos ENUM 14
Índices 15
Ciclo de vida do documento **16**
Do teclado ao banco, passo a passo 16
Abrindo um documento 16

Concorrência 16

Formato binário e modelo do documento 17

Update Yrs v1 17

Modelo Y.Doc 17

Chaves no Redis 17

NATS JetStream 18

Consultas prontas 19

Como preparar o banco 21

Com Docker (recomendado) 21

PostgreSQL próprio 21

Backup e restauração 21

Desempenho 21

Integridade, retenção e segurança 22

Papéis e menor privilégio 22

Transporte e segredos 22

Limites 22

Retenção 22

Mudanças na versão 1.3.0 23

O que muda no documento 23

Limites novos 23

Observabilidade 23

Capítulo 1

Visão geral

PostgreSQL é a fonte da verdade

O Grafo guarda três tipos de dado com necessidades diferentes, e cada um vai para a ferramenta certa. Tudo o que precisa durar (contas, equipes, arquivos, o estado do documento, versões, comentários, imagens e fontes de dados) fica no **PostgreSQL 17**. O que é passageiro e precisa ser rápido (quem está online, limites de requisição, cache do estado) fica no **Redis**. O fluxo de alterações em tempo real entre várias instâncias do servidor passa pelo **NATS JetStream**.

O editor nunca fala com o banco diretamente: tudo passa pelo servidor em Rust (Axum + Tokio), que valida, aplica e grava. Sem Postgres, Redis ou NATS configurados, o servidor sobe só em memória para desenvolvimento e avisa no log; em produção, os três são obrigatórios.

> **Regra de ouro:** o documento colaborativo é um `Y.Doc` (CRDT). O PostgreSQL guarda o **estado compactado** (`document_states`) e a **fila de updates** desde a última compactação (`document_updates`). Estado + updates na ordem de `seq` reconstroem o documento exato, byte a byte.

Camadas de armazenamento

Camada	Guarda	Durabilidade
PostgreSQL 17	Usuários, equipes, projetos, arquivos, permissões, links, estado e updates do documento, versões, comentários, assets, bibliotecas, fontes de dados	Permanente, com backup
Redis 7	Presença por arquivo, limite de requisições por IP, cache do estado do documento	Volátil (TTL); pode ser apagado a qualquer hora
NATS JetStream	Stream `GRAFO_DOCS` com os updates de cada documento, para replicar entre instâncias	Retenção limitada (24 h); o Postgres continua sendo a referência
Navegador (IndexedDB)	Cópia local do documento no modo `Grafo.html` e rascunho offline	Local ao navegador

Áreas do modelo

Área	Tabelas	Guarda
Identidade e equipes	`users` , `teams` , `team_members` , `projects`	Quem usa o Grafo e como as pessoas se organizam
Arquivos e acesso	`files` , `file_permissions` , `share_links`	Os arquivos de design e diagrama e quem pode abrir cada um
Documento colaborativo	`document_states` , `document_updates` , `versions`	O conteúdo do documento, o fluxo de alterações e as versões nomeadas
Conteúdo de apoio	`comments` , `assets` , `library_components` , `data_sources`	Comentários com respostas, imagens, componentes publicados e dados vinculados

Modo local (sem servidor)

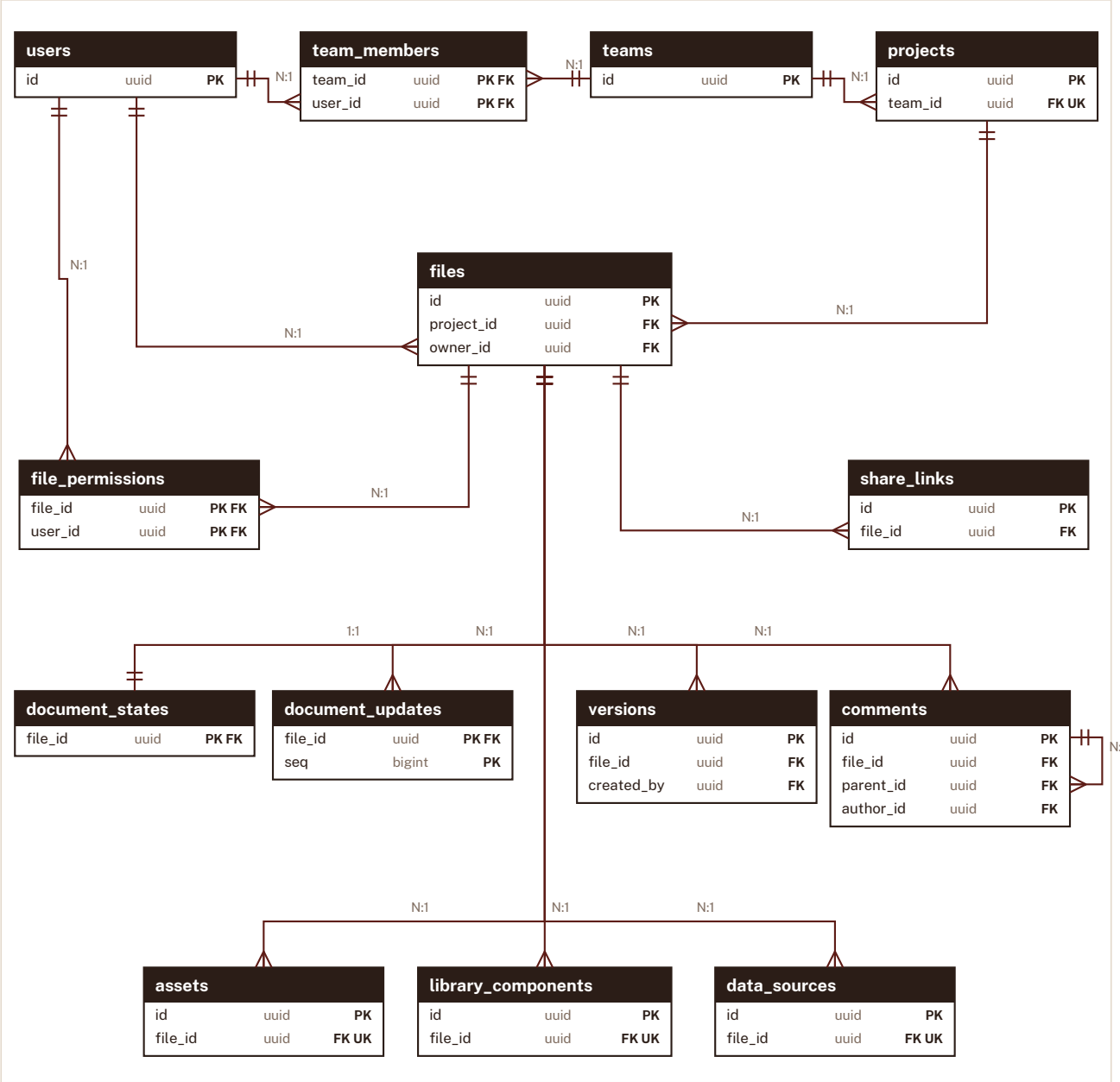
No arquivo único `Grafo.html`, o editor grava no próprio navegador, num esquema mais simples:

Onde	Chave	Conteúdo
IndexedDB `grafo`, loja `files`	`id` do arquivo	`{id, name, kind, updatedAt, thumbnail}`
IndexedDB `grafo`, loja `docs`	`id` do arquivo	Update Yjs v1 completo (`Y.encodeStateAsUpdate`)
localStorage `grafo:prefs`	—	Tema, grade, réguas, painéis abertos

Capítulo 2

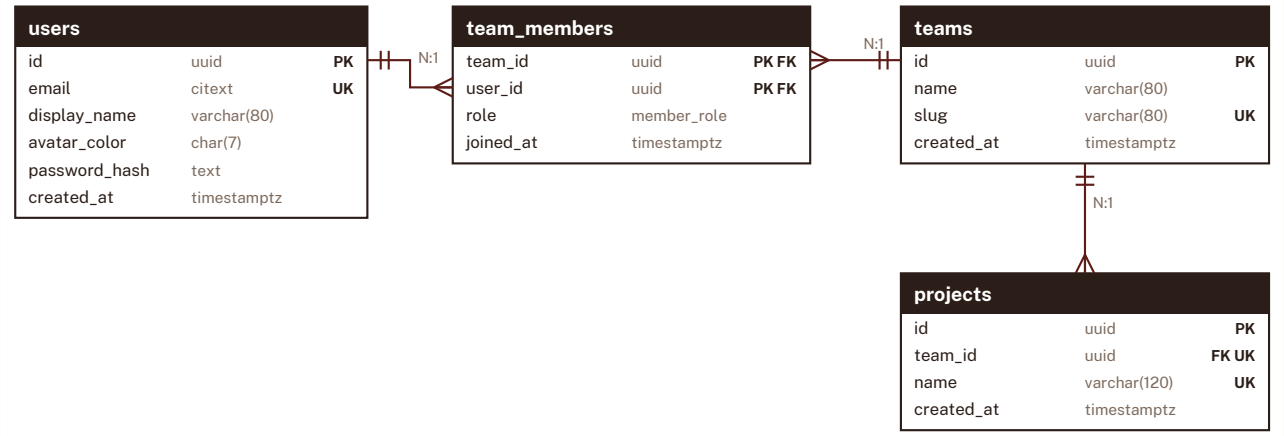
Diagrama de relacionamentos

Diagrama geral



Todas as tabelas com chaves primárias (PK), estrangeiras (FK) e únicas (UK). Pé-de-galinha = "muitos"; dois traços = "exatamente um". `versions.created_by` e `comments.author_id` também apontam para `users` (diagramas 3 e 4).

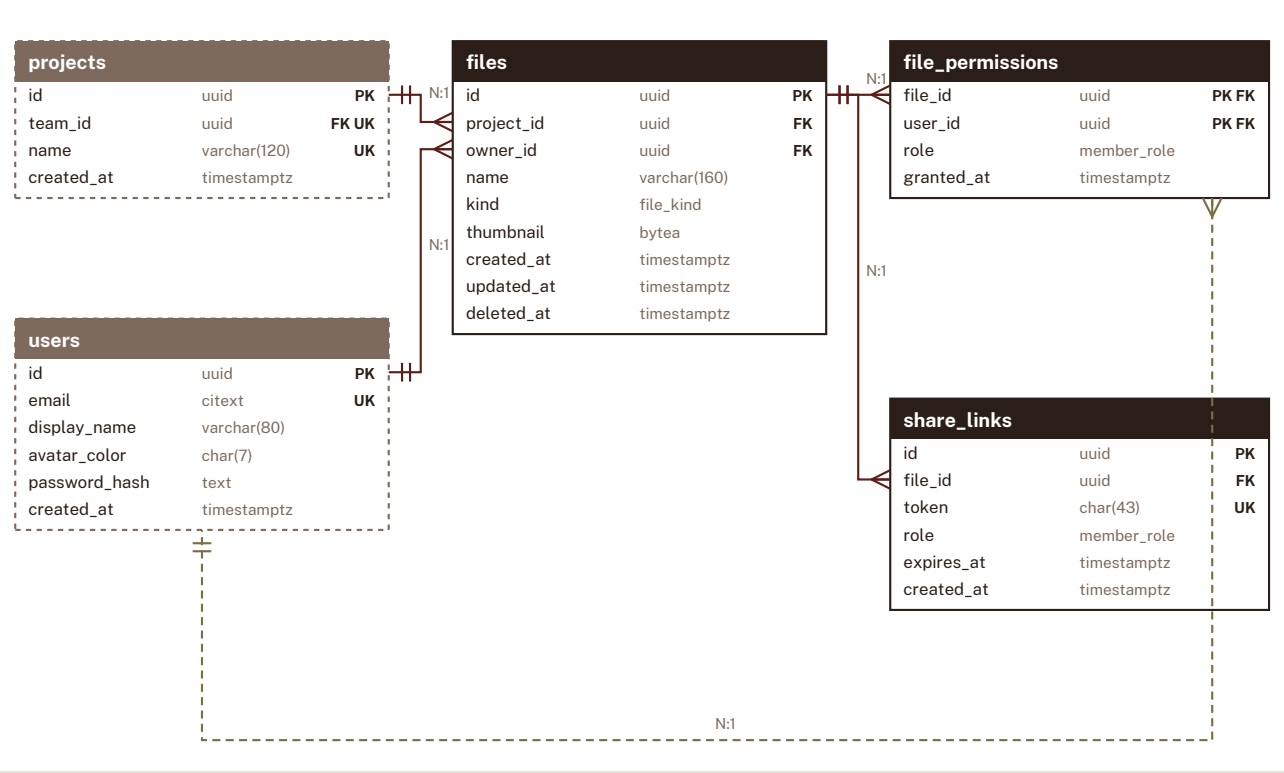
Identidade e equipes



users 1:N team_members · teams 1:N team_members · teams 1:N projects

`team_members` é a tabela de associação entre pessoas e equipes, com a chave primária composta `(team_id, user_id)` : a mesma pessoa não entra duas vezes na mesma equipe. O papel (`member_role`) define o que ela pode fazer em todos os arquivos da equipe.

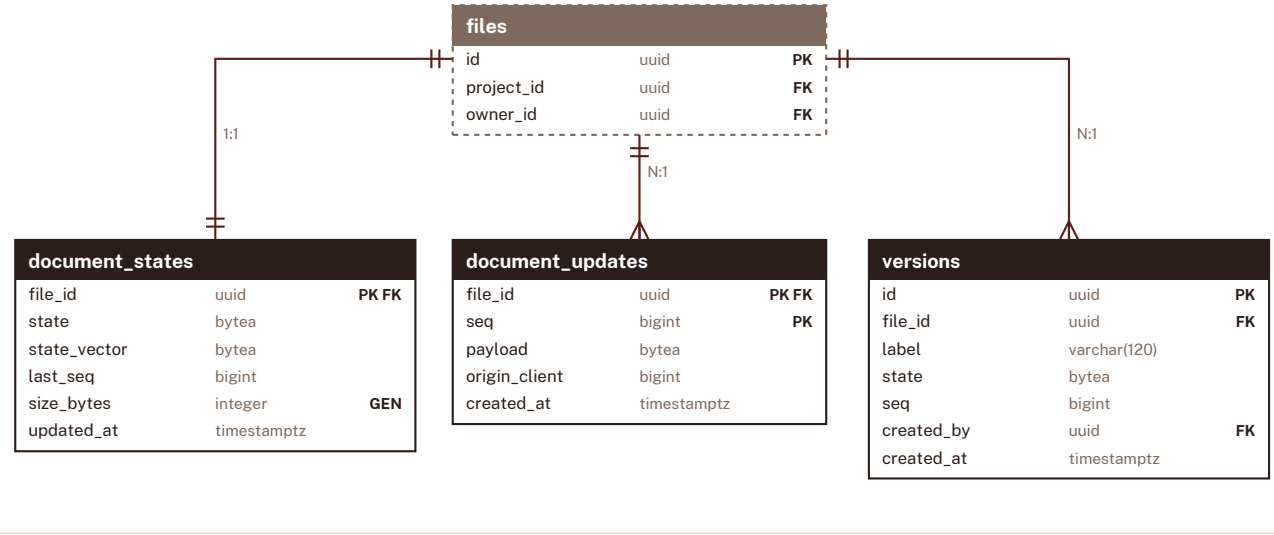
Arquivos e permissões



projects 1:N files · users 1:N files (dono) · files 1:N file_permissions · files 1:N share_links · users 1:N file_permissions (tracejado)

O acesso efetivo a um arquivo é o **maior** papel entre: dono (`files.owner_id`), papel na equipe do projeto, permissão direta (`file_permissions`) e link de compartilhamento válido (`share_links` , nunca `owner`).

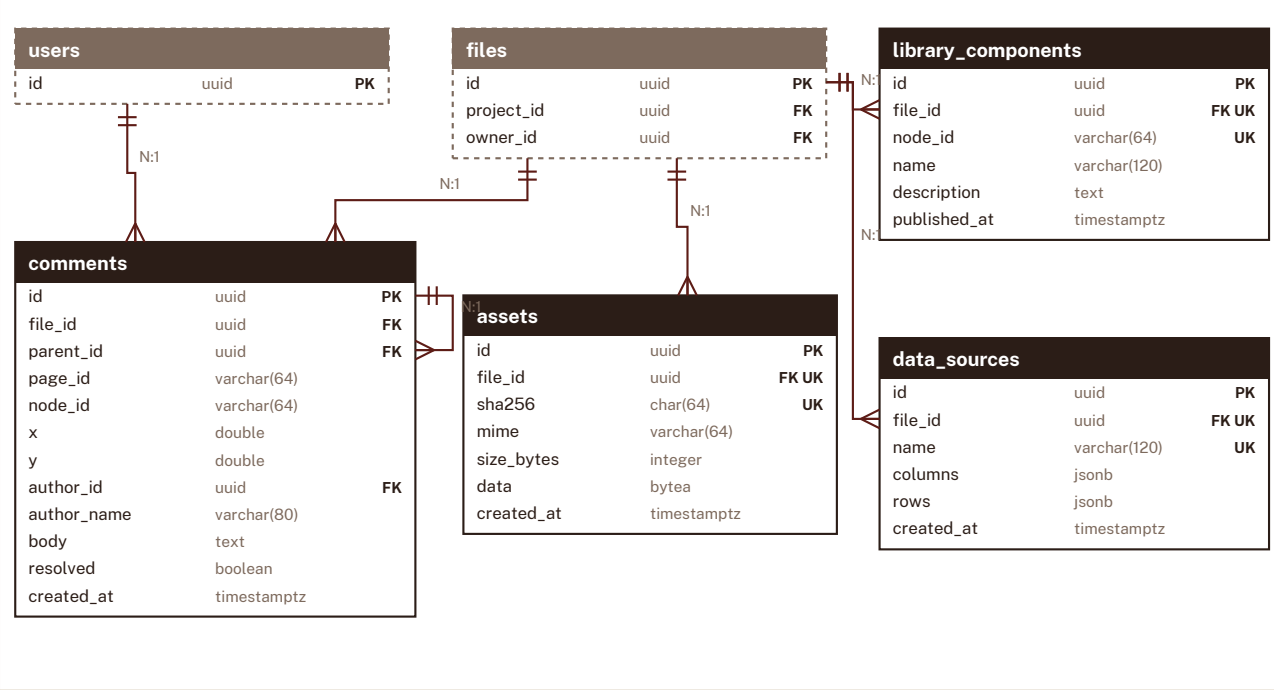
Documento colaborativo



files 1:1 document_states · files 1:N document_updates · files 1:N versions

`document_states` tem a mesma chave do arquivo (1:1). `document_updates` usa a chave composta `(file_id, seq)`, o que garante a ordem total dos updates de cada documento e impede buracos silenciosos.

Comentários, assets, bibliotecas e dados



files 1:N comments · comments 1:N comments (respostas) · files 1:N assets · files 1:N library_components · files 1:N data_sources

Capítulo 3

Regras e convenções

Convenções do modelo

- Nomes de tabelas e colunas em inglês, no plural e em `snake_case`.
- Chaves primárias `uuid` geradas no banco (`gen_random_uuid()`), exceto nas tabelas de associação e de updates, que usam chaves compostas.
- Datas sempre em `timestamptz` (UTC no banco; o editor converte para o fuso de quem vê).
- Exclusão lógica só em `files` (`deleted_at`): o arquivo some das listas, mas pode ser restaurado por 30 dias.
- Binários do documento em `bytea`, no formato Yrs/Yjs update v1, sem nenhuma transformação.
- Os ids de páginas e nós do documento (`page_id`, `node_id`) são textos curtos gerados pelo editor, não chaves do banco.

Integridade

- Toda chave estrangeira tem `ON DELETE` explícito: `CASCADE` para o que não faz sentido sem o pai (permissões, updates, versões, comentários, assets) e `SET NULL` para autoria e projeto.
- `CHECK` em cores (`^#[0-9A-Fa-f]{6}$`), slug (`^[a-z0-9-]+$`), tamanho de miniatura (512 KB), tamanho de update (8 MB), tamanho de asset (20 MB), tipos MIME aceitos, corpo de comentário (1 a 4000 caracteres) e `seq > 0`.
- `share_links.role` nunca é `owner`.
- `size_bytes` em `document_states` é coluna gerada (`GENERATED ALWAYS AS (octet_length(state)) STORED`), então nunca diverge do estado.

Validação antes de gravar

O servidor valida tudo o que chega antes de abrir a transação: UUIDs, tamanhos, MIME real do arquivo (pelos bytes, não pela extensão), nomes (1 a 160 caracteres, sem caracteres de controle) e o próprio update Yjs, que precisa ser decodificado sem erro pelo Yrs. Um update inválido é recusado e a conexão recebe o estado correto de volta.

Capítulo 4

Tabelas

Tipos conforme o PostgreSQL. Na coluna Nulo, "sim" indica que o campo aceita `NULL`.

users

Contas de quem usa o Grafo.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária (`gen_random_uuid()`)
`email`	`citext`	não	E-mail, único sem diferenciar maiúsculas
`display_name`	`varchar(80)`	não	Nome mostrado nos cursores e comentários
`avatar_color`	`char(7)`	não	Cor do cursor (padrão `#5D1C16`)
`password_hash`	`text`	sim	Hash Argon2id; nulo para contas de link
`created_at`	`timestamptz`	não	Criação

teams

Equipes que reúnem pessoas e projetos.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`name`	`varchar(80)`	não	Nome da equipe
`slug`	`varchar(80)`	não	Identificador na URL, único, `[a-z0-9-]`
`created_at`	`timestamptz`	não	Criação

team_members

Quem faz parte de cada equipe e com qual papel.

Campo	Tipo	Nulo	Descrição
`team_id`	`uuid`	não	Equipe (PK, FK `teams`, cascade)
`user_id`	`uuid`	não	Pessoa (PK, FK `users`, cascade)
`role`	`member_role`	não	Papel (padrão `editor`)
`joined_at`	`timestamptz`	não	Entrada na equipe

projects

Pastas de arquivos dentro de uma equipe.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`team_id`	`uuid`	não	Equipe (FK, cascade)
`name`	`varchar(120)`	não	Nome; único dentro da equipe
`created_at`	`timestamptz`	não	Criação

files

Cada arquivo de design, diagrama ou quadro.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária; é também o nome da sala do WebSocket
`project_id`	`uuid`	sim	Projeto (FK, set null); nulo = rascunhos
`owner_id`	`uuid`	sim	Dono (FK, set null)
`name`	`varchar(160)`	não	Nome mostrado na lista
`kind`	`file_kind`	não	`design` , `diagram` ou `board`
`thumbnail`	`bytea`	sim	PNG de até 512 KB gerado pelo editor
`created_at`	`timestamptz`	não	Criação
`updated_at`	`timestamptz`	não	Última alteração (índice `files_updated_idx`)
`deleted_at`	`timestamptz`	sim	Exclusão lógica

file_permissions

Permissões diretas de uma pessoa num arquivo.

Campo	Tipo	Nulo	Descrição
`file_id`	`uuid`	não	Arquivo (PK, FK, cascade)
`user_id`	`uuid`	não	Pessoa (PK, FK, cascade)
`role`	`member_role`	não	Papel concedido
`granted_at`	`timestamptz`	não	Quando foi concedido

share_links

Links de compartilhamento com papel e validade.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo (FK, cascade)
`token`	`char(43)`	não	32 bytes aleatórios em base64url; único
`role`	`member_role`	não	Papel dado pelo link (nunca `owner`)
`expires_at`	`timestamptz`	sim	Validade; nulo = sem prazo
`created_at`	`timestamptz`	não	Criação

document_states

Estado compactado do documento colaborativo.

Campo	Tipo	Nulo	Descrição
`file_id`	`uuid`	não	Arquivo (PK, FK, cascade); relação 1:1
`state`	`bytea`	não	Update Yrs v1 com o documento inteiro
`state_vector`	`bytea`	não	Vetor de estado do mesmo momento
`last_seq`	`bigint`	não	Último `seq` incluído na compactação
`size_bytes`	`integer`	não	Gerado: `octet_length(state)`
`updated_at`	`timestamptz`	não	Última compactação

document_updates

Updates recebidos desde a última compactação.

Campo	Tipo	Nulo	Descrição
`file_id`	`uuid`	não	Arquivo (PK, FK, cascade)
`seq`	`bigint`	não	Número sequencial por arquivo (PK, `> 0`)
`payload`	`bytea`	não	Update Yrs v1 (até 8 MB)
`origin_client`	`bigint`	sim	`clientID` do Yjs que gerou o update
`created_at`	`timestamptz`	não	Recebimento

versions

Versões nomeadas (histórico de versões).

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo (FK, cascade)
`label`	`varchar(120)`	não	Nome da versão ("Revisão do cliente")
`state`	`bytea`	não	Estado completo Yrs v1 no momento
`seq`	`bigint`	não	`seq` do documento quando foi criada
`created_by`	`uuid`	sim	Autor (FK `users`, set null)
`created_at`	`timestamptz`	não	Criação (índice por arquivo e data)

comments

Comentários e respostas presos a pontos do canvas.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo (FK, cascade)
`parent_id`	`uuid`	sim	Comentário pai (FK própria, cascade); nulo = início da conversa
`page_id`	`varchar(64)`	não	Página do documento
`node_id`	`varchar(64)`	sim	Nó ao qual o comentário está preso
`x`	`double precision`	não	Posição no canvas
`y`	`double precision`	não	Posição no canvas
`author_id`	`uuid`	sim	Autor (FK, set null)
`author_name`	`varchar(80)`	não	Nome guardado, para manter o histórico
`body`	`text`	não	Texto (1 a 4000 caracteres)
`resolved`	`boolean`	não	Conversa resolvida
`created_at`	`timestamptz`	não	Criação

assets

Imagens usadas nos documentos, sem duplicar.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo (FK, cascade)
`sha256`	`char(64)`	não	Hash do conteúdo; único por arquivo
`mime`	`varchar(64)`	não	PNG, JPEG, WebP, GIF ou SVG

Campo	Tipo	Nulo	Descrição
`size_bytes`	`integer`	não	1 byte a 20 MB
`data`	`bytea`	não	Conteúdo
`created_at`	`timestamptz`	não	Envio

library_components

Componentes publicados como biblioteca da equipe.

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo de origem (FK, cascade)
`node_id`	`varchar(64)`	não	Nó do componente; único por arquivo
`name`	`varchar(120)`	não	Nome na biblioteca
`description`	`text`	não	Descrição (padrão vazio)
`published_at`	`timestamptz`	não	Publicação

data_sources

Tabelas de dados vinculadas às formas (CSV).

Campo	Tipo	Nulo	Descrição
`id`	`uuid`	não	Chave primária
`file_id`	`uuid`	não	Arquivo (FK, cascade)
`name`	`varchar(120)`	não	Nome; único por arquivo
`columns`	`jsonb`	não	Lista de colunas (`array`)
`rows`	`jsonb`	não	Lista de linhas (`array` de objetos)
`created_at`	`timestamptz`	não	Importação

Tipos ENUM

Tipo	Valores	Uso
member_role	`owner` , `editor` , `commenter` , `viewer`	Papéis em equipes, permissões e links. `commenter` comenta, mas não edita.
file_kind	`design` , `diagram` , `board`	Tipo do arquivo; muda a biblioteca aberta por padrão no editor.

Índices

Índice	Tabela	Para quê
`files_project_idx`	files (project_id) parcial	Listar arquivos de um projeto, ignorando os excluídos
`files_updated_idx`	files (updated_at DESC) parcial	Lista "Recentes"
`versions_file_idx`	versions (file_id, created_at DESC)	Histórico de versões
`comments_file_idx`	comments (file_id, created_at)	Abrir os comentários de um arquivo
`Chaves e UNIQUE`	todas	Cada PK e UNIQUE cria o próprio índice B-tree

Capítulo 5

Ciclo de vida do documento

Do teclado ao banco, passo a passo

Passo	Onde	O que acontece
1	Editor	Cada alteração vira uma transação Yjs; o `WebsocketProvider` envia a mensagem `sync/update` (tipo 0, subtipo 2).
2	Servidor	Confere o tamanho (`GRAFO_MAX_UPDATE_BYTES`), decodifica com Yrs e aplica no `Doc` em memória da sala.
3	Servidor	Reenvia o update para os outros clientes da sala e publica em `grafo.docs.{file_id}.updates` no JetStream.
4	PostgreSQL	Grava em `document_updates` com o próximo `seq` do arquivo, na mesma transação que atualiza `files.updated_at` .
5	Outras instâncias	Consomem o subject, ignoram o que elas mesmas publicaram (cabeçalho de instância) e aplicam o update.
6	Compactação	A cada `GRAFO_SNAPSHOT_EVERY` updates (200), grava o estado inteiro em `document_states` e apaga os updates com `seq <= last_seq` .
7	Versão	"Salvar versão" copia o estado atual para `versions` com um nome; restaurar aplica a diferença como um update novo, sem apagar a história.

Abrindo um documento

- O servidor procura o estado no Redis (`grafo:state:{file_id}`); se não houver, lê `document_states` e aplica os `document_updates` com `seq > last_seq` , em ordem.
- O cliente envia o próprio vetor de estado (`sync step 1`); o servidor responde só com o que falta (`sync step 2`).
- Sala sem ninguém por 60 s é compactada e descarregada da memória.

Concorrência

Não há travas de linha nem "última gravação vence": o Yjs é um CRDT, e qualquer ordem de aplicação dos updates converge para o mesmo documento. O `seq` serve à durabilidade e à reconstrução, não à resolução de conflitos. A compactação usa `SELECT ... FOR UPDATE` na linha de `document_states` para que duas instâncias não compactem o mesmo arquivo ao mesmo tempo.

Capítulo 6

Formato binário e modelo do documento

Update Yrs v1

`state`, `payload` e `versions.state` guardam bytes no formato de update v1 do Yjs, que o Yrs (Rust) e o Yjs (JavaScript) leem e escrevem igual. Um update é uma lista de estruturas por cliente (`clientID`, relógio, conteúdo) seguida do conjunto de exclusões. O vetor de estado é um mapa `clientID → relógio` codificado com inteiros variáveis (varint).

```
let update = Y.encodeStateAsUpdate(doc)
let vector = Y.encodeStateVector(doc)
let diff = Y.encodeStateAsUpdate(doc, remoteVector)
Y.applyUpdate(otherDoc, update)
```

Modelo Y.Doc

Raiz	Tipo	Conteúdo
`meta`	Y.Map	`name`, `kind`, `version`
`pages`	Y.Array	Ids das páginas, na ordem das abas
`pageInfo`	Y.Map	id → `{id, name, bg}`
`nodes`	Y.Map de Y.Map	id → propriedades do nó (posição, tamanho, estilo, texto, conexões…)
`comments`	Y.Map	id → `{id, page, x, y, author, color, body, replies, resolved, createdAt}`
`styles`	Y.Map	id → `{id, kind, name, value}` (cor, texto, efeito)
`dataSources`	Y.Map	nome → `{columns, rows}`

Tipos de nó: `frame`, `section`, `group`, `rect`, `ellipse`, `line`, `arrow`, `polygon`, `star`, `path`, `text`, `image`, `sticky`, `shape`, `connector`, `table`, `component`, `instance` e `boolean`. A ordem de desenho vem do campo `z` (número real), o que permite inserir entre dois nós sem renumerar os outros.

Chaves no Redis

Chave	Tipo	TTL	Conteúdo
`grafo:presence:{file_id}`	hash	60 s	`clientID` → JSON `{name, color, page, cursor}`
`grafo:rl:{ip}:{minuto}`	string	120 s	Contador de requisições (`INCR`), limite `GRAFO_RATE_LIMIT`
`grafo:state:{file_id}`	string	10 min	Estado Yrs v1 mais recente, para abrir rápido

NATS JetStream

Item	Valor
Stream	`GRAFO_DOCS`, subjects `grafo.docs.*.updates`, retenção por limites, `max_age` 24 h, armazenamento em arquivo
Subject por arquivo	`grafo.docs.{file_id}.updates` (payload = update Yrs v1)
Cabeçalhos	`Grafo-Instance` (id da instância, para ignorar o próprio eco) e `Nats-Msg-Id` (deduplicação)
Consumidor	Efêmero por sala aberta, `deliver_policy = new`

Capítulo 7

Consultas prontas

Arquivos mais recentes de uma equipe

```
SELECT f.id, f.name, f.kind, f.updated_at
FROM files f
JOIN projects p ON p.id = f.project_id
WHERE p.team_id = $1 AND f.deleted_at IS NULL
ORDER BY f.updated_at DESC
LIMIT 50;
```

Papel efetivo de uma pessoa num arquivo

```
SELECT max(r) FROM (
  SELECT 'owner'::member_role AS r FROM files WHERE id = $1 AND owner_id = $2
  UNION ALL SELECT tm.role FROM files f JOIN projects p ON p.id = f.project_id
    JOIN team_members tm ON tm.team_id = p.team_id AND tm.user_id = $2 WHERE f.id = $1
  UNION ALL SELECT role FROM file_permissions WHERE file_id = $1 AND user_id = $2
) x;
```

Reconstruir um documento

```
SELECT state, last_seq FROM document_states WHERE file_id = $1;
SELECT payload FROM document_updates
WHERE file_id = $1 AND seq > $2
ORDER BY seq;
```

Próximo seq e gravação de um update

```
INSERT INTO document_updates (file_id, seq, payload, origin_client)
SELECT $1, coalesce(max(seq), 0) + 1, $2, $3
FROM document_updates WHERE file_id = $1
RETURNING seq;
```

Espaço ocupado por arquivo

```
SELECT f.name, s.size_bytes,
       (SELECT coalesce(sum(octet_length(payload)), 0) FROM document_updates u WHERE u.file_id = f.id) A
S pending,
       (SELECT coalesce(sum(size_bytes), 0) FROM assets a WHERE a.file_id = f.id) AS assets
FROM files f LEFT JOIN document_states s ON s.file_id = f.id
ORDER BY s.size_bytes DESC NULLS LAST;
```

Comentários abertos com respostas

```
SELECT c.id, c.author_name, c.body, count(r.id) AS replies
FROM comments c LEFT JOIN comments r ON r.parent_id = c.id
WHERE c.file_id = $1 AND c.parent_id IS NULL AND NOT c.resolved
GROUP BY c.id ORDER BY c.created_at;
```

Limpeza de arquivos excluídos há mais de 30 dias

```
DELETE FROM files WHERE deleted_at < now() - interval '30 days';
```

Capítulo 8

Como preparar o banco

Com Docker (recomendado)

O `docker-compose.yml` sobe PostgreSQL 17, Redis 7, NATS com JetStream e o servidor. As migrações em `server/migrations` rodam sozinhas na primeira inicialização do servidor.

```
cp .env.example .env
docker compose up -d
docker compose logs -f server
```

PostgreSQL próprio

```
CREATE ROLE grafo_owner NOLOGIN;
CREATE ROLE grafo_app LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE grafo OWNER grafo_owner;
\c grafo
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;
GRANT CONNECT ON DATABASE grafo TO grafo_app;
GRANT USAGE ON SCHEMA public TO grafo_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO grafo_app;
ALTER DEFAULT PRIVILEGES FOR ROLE grafo_owner IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO grafo_app;
```

Depois, aponte `DATABASE_URL=postgres://grafo_app:...@host:5432/grafo?sslmode=require` e rode as migrações com o papel dono (`psql -f server/migrations/0001_init.sql`).

Backup e restauração

```
pg_dump -Fc -d grafo -f grafo-$(date +%F).dump
pg_restore -d grafo_novo --no-owner grafo-2026-10-24.dump
```

Faça o backup depois de uma compactação para que ele fique menor: como o estado e os updates estão no mesmo banco, um `pg_dump` consistente basta. Redis e NATS não precisam de backup: são reconstruídos a partir do PostgreSQL.

Desempenho

- `shared_buffers` em 25% da RAM e `effective_cache_size` em 50–75%.
- `document_updates` recebe muitas inserções pequenas: mantenha o autovacuum ativo e, em cargas altas, use `fillfactor = 90` nessa tabela.
- Colunas `bytea` grandes vão para o TOAST com compressão `lz4` (`ALTER TABLE ... ALTER COLUMN state SET COMPRESSION lz4`).
- Use um pool (o servidor usa até 16 conexões por instância).

Capítulo 9

Integridade, retenção e segurança

Papéis e menor privilégio

Papel	Pode	Não pode
`grafo_owner`	Criar e alterar tabelas (migrações)	Fazer login
`grafo_app`	Ler e gravar dados	Criar, alterar ou apagar tabelas
`grafo_readonly`	Consultas de relatório e backup	Gravar

Transporte e segredos

- Conexões com `sslmode=require` (ou `verify-full`) fora da rede do Docker; Redis com senha e TLS; NATS com usuário e TLS.
- Senhas e tokens só em variáveis de ambiente, nunca no repositório (`.env` está no `.gitignore`).
- Senhas de usuário em Argon2id; tokens de link com 256 bits aleatórios.

Limites

Limite	Valor	Onde é conferido
Update do documento	8 MB	Servidor e `CHECK` em `document_updates`
Miniatura	512 KB	`CHECK` em `files`
Asset	20 MB	Servidor e `CHECK` em `assets`
Comentário	1 a 4000 caracteres	`CHECK` em `comments`
Requisições	120 por minuto por IP	Redis (`grafo:rl:*`)

Retenção

Dado	Fica por
`document_updates`	Até a próxima compactação (no máximo 200 updates)
`versions`	Para sempre, até alguém apagar
Arquivo excluído	30 dias com `deleted_at`, depois é apagado com tudo o que depende dele
Stream do JetStream	24 horas
Presença e cache no Redis	60 s e 10 min

Capítulo 10

Mudanças na versão 1.3.0

A versão 1.3.0 não muda o esquema do PostgreSQL: nenhuma tabela, coluna, índice, tipo ENUM ou migração nova. Quem já roda o Grafo atualiza o servidor sem tocar no banco, e as chaves do Redis e os subjects do NATS continuam os mesmos.

O que muda no documento

As novidades do editor ficam dentro do documento colaborativo (o mapa `nodes` do Y.Doc), que o servidor guarda como binário Yrs em `document_states` e `document_updates`. São propriedades opcionais: versões anteriores do editor simplesmente as ignoram.

Propriedade	Em qual nó	Para quê
`table: { rows, cols, header }`	frame da tabela	Lembra o tamanho e se a primeira linha é cabeçalho
`cell: { row, col }`	shape de cada célula	Posição da célula dentro da tabela

O diagrama a partir de texto e o estilo rápido reaproveitam propriedades que já existiam: `dash` nos conectores tracejados de um Mermaid `-.->` e `textColor`, `fill` e `stroke` nas formas.

O diagrama a partir de texto, a tabela gerada de um CSV e a busca com substituição só criam ou alteram nós comuns (`shape`, `section`, `connector` e `frame`), sempre dentro de uma transação, então chegam aos outros participantes como um único update e são desfeitos com um Ctrl+Z. O texto digitado na janela do diagrama fica só no navegador (`localStorage`) e não vai para o banco.

Limites novos

Operação	Limite	Por quê
Diagrama a partir de texto	400 formas e 800 conectores; rótulos de até 300 caracteres; campo de até 50.000 caracteres	Mantém o update de inserção pequeno e o layout instantâneo
Inserir tabela	200 linhas × 24 colunas	No máximo 4.800 células num único update
Tabela a partir de CSV	99 linhas + cabeçalho, 24 colunas	A fonte de dados continua inteira em data_sources
Substituir texto	Até 20.000 caracteres por camada; camadas bloqueadas ficam de fora	Respeita o bloqueio e o tamanho máximo de texto do editor

Observabilidade

`GET /api/metrics` expõe no formato do Prometheus o estado do servidor sem consultar o banco: `grafo_dependency_up{service="postgres"}` indica se o pool do PostgreSQL está aberto, e os contadores `grafo_updates_applied_total` e `grafo_update_bytes_total` mostram o volume que segue para `document_updates` entre as compactações. Com `GRAFO_API_TOKEN` definido, a rota exige o token.

Uma conta rápida para dimensionar o disco: bytes de updates por dia ÷ GRAFO_SNAPSHOT_EVERY dá uma ideia de quantas compactações acontecem, e cada compactação troca as linhas de document_updates por um único estado em document_states.